INDIVIDUAL LABORATORY REPORT

DevOps Lab
Project Report

Linux | Git & GitHub | Docker | Jenkins

STUDENT

Alisher Panabaev
Group It2-2308 | Student ID 37332

PROJECT STATUS

Application and Docker workflow verified. Jenkins controller is running; first-run setup, job creation, and the actual Pipeline build are pending local confirmation.

19 October 2026

Repository: github.com/alisherqa06/panabaev-alisher-devops

1. Student Information

This report documents the existing Linux project and the implementation completed in the repository. It distinguishes verified command output from Jenkins steps that still require the local administrator setup.

Name	Alisher
Surname	Panabaev
Group	It2-2308
Student ID	37332
Course	DevOps
Linux project	`/home/acer/devops/panabaev_alisher`

2. Linux

The existing project runs in Ubuntu 26.04.1 LTS under WSL 2. The WSL status check showed Ubuntu running with version 2 and the Docker Desktop internal distribution running with version 2. The project directory contains app, scripts, config, logs, and backup directories.

The Bash script `scripts/Panabaev_Alisher_system.sh` resolves the student information file relative to its own location. A syntax check and actual run succeeded. Its output included the student record, Linux user acer, hostname KOMPUTER, date, OS release, disk use, memory use, and the successful existence check for the student information file.

```
bash -n scripts/Panabaev_Alisher_system.sh
./scripts/Panabaev_Alisher_system.sh
stat -c '%a %n' private.txt public.txt scripts/Panabaev_Alisher_system.sh
```

Verified permissions were 600 for private.txt, 644 for public.txt, and 755 for the Bash script. The log file had two ERROR entries and two WARNING entries; it remains excluded from Git by the *.log rule. The ignored backup directory and .env rule were also checked.

```
find logs -maxdepth 1 -type f -name '*.log'
grep -c ERROR logs/application.log
grep -c WARNING logs/application.log
head -n 2 logs/application.log
tail -n 2 logs/application.log
ps aux | head
```

Linux provides the shell and standard system tools used in the lab for file operations, permissions, process inspection, system information, and log searches.

3. Git & GitHub

The existing Git history was preserved. The original five commits remain in order, and the new implementation was committed as `464bef37965c63d4a6d4fa5e6a67b1274e24fa50` (Add Docker app and Jenkins pipeline). The commit was pushed to the existing main branch. development was fast-forwarded to the same commit; both branches were verified on the remote.

```
git status --short --branch
git log --oneline --decorate -7
git ls-remote origin refs/heads/main refs/heads/development
```

Git records and identifies source revisions. GitHub hosts the remote repository and makes a selected revision available to Jenkins. Jenkins uses a committed revision so the build can be traced and repeated from the same source state.

Repository: https://github.com/alisherqa06/panabaev-alisher-devops

4. Docker

Docker Desktop is the Linux container engine for the Windows 11 and WSL 2 setup. In Ubuntu, Docker CLI 29.8.2 and Docker Compose 5.6.0 were available, `docker info` reached the Desktop engine, and the requested hello-world image ran successfully.

The app is a standard-library Python HTTP server. It displays the four student values from environment variables, falls back to the personalized defaults when values are missing or empty, serves the page on port 8080, and provides `/healthz` for readiness checks. HTML output escapes student values.

The Dockerfile uses FROM, WORKDIR, COPY, RUN, EXPOSE, and CMD. It creates a non-root app user. The image name is `panabaev-alisher-devops`; the required container name is `panabaev-alisher-container`. The image is the reusable package, while the container is an instance running its process and environment.

```
docker build -t panabaev-alisher-devops .
docker run -d --name panabaev-alisher-container \
  -p 8080:8080 -e STUDENT_NAME=Alisher \
  -e STUDENT_SURNAME=Panabaev -e STUDENT_GROUP=It2-2308 \
  -e STUDENT_ID=37332 panabaev-alisher-devops
python3 scripts/check_http.py http://127.0.0.1:8080
```

Four unit tests passed: personalized defaults, environment overrides, HTML escaping, and the live health/student-page endpoints. The HTTP smoke check confirmed the expected name, surname, group, and ID. The container logs showed those values, docker exec succeeded, and stop/start and removal were checked.

Docker Compose also completed up --build -d, ps, logs, and down. The HTTP check passed while the Compose container was running. The same image and container names are configured in docker-compose.yml.

```
docker compose up --build -d
docker compose ps
docker compose logs
docker compose down
```

5. Jenkins

A custom Jenkins LTS controller image was built with Python 3.13.5, Git 2.47.3, and Docker CLI 29.8.2. The controller runs in Docker Desktop on port 8081. A check from inside the Jenkins image confirmed that its Docker CLI reaches the same Docker Desktop engine through the mounted WSL socket; the Jenkins login page returned HTTP 200.

Jenkins is not yet fully configured. Its first-run unlock and plugin setup requires a local browser action and an administrator account. The exact job has not yet been created, so no Jenkins build result or Finished: SUCCESS is claimed.

After the local setup wizard is completed, create the Pipeline item Panabaev_Alisher_DevOps and configure Pipeline script from SCM with Git, repository URL `https://github.com/alisherqa06/panabaev-alisher-devops`, branch `*/main`, and script path `Jenkinsfile`. If GitHub access is private, add credentials only in Jenkins. Then choose Build Now.

The committed Jenkinsfile defines these five stages in order:

Checkout	Checks out the configured SCM source, prints student environment values, and records the source commit.
Build	Compiles the Python app and HTTP smoke-check script.
Test	Runs the standard-library unittest suite.
Docker Build	Builds panabaev-alisher-devops.
Docker Run	Removes an old named container if present, starts the app with the required values, waits for HTTP readiness, checks the page, and prints logs.

Pipeline commands are fail-fast: a failed required command returns a non-zero status, stops later stages, and marks that build unsuccessful. The success state must be verified in the actual Jenkins build page and Console Output.

6. Integrated DevOps Workflow

The intended flow is Linux -> Git -> GitHub -> Jenkins -> Build -> Test -> Docker Build -> Docker Run -> SUCCESS.

Linux	Provides the shell and system utilities used to develop and inspect the project.
Git	Tracks changes, commits, branches, and the exact source revision.
GitHub	Stores the remote repository and lets Jenkins fetch the committed source.
Jenkins checkout	Uses the configured GitHub branch and records the source commit used by a build.
Docker image	Packages the application and its runtime into a reusable artifact.
Docker container	Runs an instance of that image with student environment values and published port 8080.
Pipeline failure	Stops subsequent stages and records the failing command output; it cannot be called successful.

The app and Docker stages of this workflow have been executed successfully from Ubuntu. The final Jenkins-triggered portion remains pending the administrator setup and actual Pipeline build.

7. Screenshots

No screenshot has been invented or inserted. Capture the following from the actual desktop after the Jenkins build is successful, then add them to the report before hand-in:

- Docker Desktop Settings showing Resources > WSL Integration with Ubuntu enabled, if this screen is available.

- Jenkins Dashboard showing the running local controller.

- Job Panabaev_Alisher_DevOps showing all five Pipeline stages.

- Console Output with student values, source commit, and the actual Finished: SUCCESS line.

- Docker Desktop or Ubuntu terminal showing image panabaev-alisher-devops and container panabaev-alisher-container.

- Windows browser showing the student page at http://localhost:8080 and the GitHub commit page.

Keep the screenshots free of passwords, tokens, and other secrets.

8. Conclusion

The original Linux repository and commit history were preserved. The student HTTP application, tests, Docker image, named container, Compose configuration, Jenkinsfile, Jenkins controller image, README, video script, and hand-in checklist are present in the repository. The Docker workflow and unit tests passed in the actual WSL 2 environment.

The project is not yet ready for final submission because the Jenkins setup wizard, exact job creation, successful Pipeline execution, and real screenshots remain outstanding. Complete those steps and update this report with the actual build number and screenshots; do not submit this status version as evidence of a successful Jenkins run.

Video demonstration: record one continuous 10-20 minute screen-and-voice walkthrough as the student, following `docs/video-demo-script-ru.md`.

References

Docker Docs, Docker Desktop WSL 2 backend: https://docs.docker.com/desktop/features/wsl

Jenkins Handbook, Getting started with Pipeline: https://www.jenkins.io/doc/book/pipeline/getting-started/

Jenkins Handbook, Docker: https://www.jenkins.io/doc/book/installing/docker/